UTF-16 retains the two-byte format for the first 65,536 codepoints of Unicode/UCS and a pair of two-byte units for the rest, making it a variable-width code unit. The proposed, fixed-width four-byte system UCS-4 (ISO-10646) is still kicking around today as UTF-32. The main argument for retaining a fixed-width encoding is that some algorithms for string processing are far faster for fixed-width data (there's no need to evaluate every code unit when finding the nth character, for example, making a UTF-32 implementation an *O(1)* problem vs. *O(n)* for variable-width).

UTF-32 does see some use for internal application storage on Unix and Linux systems but is rarely encountered in, say, the world of web development. The argument concerning fixed-size characters is also debatable when one considers the fact that some Unicode characters are composed of multiple codepoints. The diacritical marks mentioned earlier combine base letters with one (or more) accent characters (e.g., "a" + "`" to make "à"), and some of the recently added emoji alphabets use multi-codepoint characters: "Man: Bald" 👨‍🦲 is composed of "Man" 👨, "Zero Width Joiner", and "Emoji Component Bald" 🦲! Take a look at the following:

```
var_dump("\u{1F468}\u{200D}\u{1F9B2}");
```

```
Output:
string(11) "👨‍🦲"
```

Now, a word of warning. Not all systems perfectly implement the Unicode standard, and there are dozens of versions of the Unicode standard. Ideally when you run the code above, you'll get a bald chap appearing. My MacBook Pro running the latest MacOS renders correctly, but a 10-year-old unpatched Linux distro might well show the separate "Man" and "Emoji Component Bald" glyphs (and possibly a numeric value for the non-rendering joiner). That's one of the points this chapter tries to make: encoding is hard, maintaining software as encoding standards change is even harder. We'll have some more multi-codepoint fun (with flags) later. Anyway, back to our example.

The code above uses a special string syntax called a *Unicode escape sequence*. When the sequence `\u{XXXX}` is used in a double-quoted string, it will be substituted for the Unicode character at codepoint `XXXX`. `\u{1F468}` is the "Man" emoji character at codepoint *1F468*, *200D* is a special non-printing character used to join *1F468* and *1F9B2* ("Emoji Component Bald") to give us our follically challenged, male-identifying human. If your terminal software is Unicode-compliant (and your system has the necessary font installed), you should see the correct emoji.